

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«Казанский (Приволжский) федеральный университет»

ИНСТИТУТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

Кафедра прикладной математики и искусственного интеллекта

Направление: 01.03.04 — Прикладная математика

Профиль: прикладная математика

КУРСОВАЯ РАБОТА

Сравнение методов декомпозиции области при решении уравнения энергии

Студент 3 курса

группы 09-321

«_____» _____ 2026 г.

Лещенко И.Ф.

Научный руководитель

старший преподаватель

«_____» _____ 2026 г.

Федотов П.Е.

Заведующий кафедрой

к.ф.-м.н., доцент

«_____» _____ 2026 г.

Тумаков Д.Н.

Казань — 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Постановка задачи	5
1.1 Краевая задача и разностная аппроксимация	5
1.2 Двумерная постановка	6
1.3 Идея метода Шварца	7
1.4 Алгоритмы	8
1.5 Модельный пример	10
2 Практическая часть	11
2.1 Программный комплекс	11
2.2 Одномерная задача	11
2.3 Размер области пересечения	13
2.4 Распределение узлов сетки	14
2.5 Количество подобластей	14
2.6 Метод Зейделя как локальный решатель	16
2.7 Двумерная задача	17
2.8 Сводка результатов	18
ЗАКЛЮЧЕНИЕ	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	21
ПРИЛОЖЕНИЕ А (обязательное)	23
ПРИЛОЖЕНИЕ Б (обязательное)	24
ПРИЛОЖЕНИЕ В (обязательное)	25
ПРИЛОЖЕНИЕ Г (обязательное)	26

ВВЕДЕНИЕ

Задачи, в основе которых лежат уравнения эллиптического типа, являются почти в каждой инженерной дисциплине. К ним сводятся расчёты температурных полей в конструкциях, оценка распределения потенциала в проводящих средах, моделирование процессов фильтрации и множество других прикладных задач [1]. Аналитическое выражение решения удаётся получить лишь в ограниченном числе случаев — обычно при существенной идеализации геометрии и упрощённой форме правой части. Поэтому в практической работе приходится опираться на численные методы: разностные схемы, конечные элементы, итерационные алгоритмы решения возникающих систем [2].

С увеличением размерности сеточной задачи определяющим параметром становится не столько достижимая точность, сколько объём требуемых вычислений. Решение системы линейных алгебраических уравнений, к которой сводится разностная аппроксимация, при густой сетке превращается в основную статью расходов вычислительного времени. Один из путей преодоления этого ограничения — метод декомпозиции области: исходная задача заменяется набором меньших по размеру подзадач, решаемых на отдельных подобластях [3]. В идеальной ситуации часть подзадач выполняется параллельно, что позволяет приблизить общее время вычислений к времени решения одной локальной задачи.

Сама идея относится к XIX веку: первый вариант альтернирующего метода предложил Г. А. Шварц [4]. В современной литературе устоявшимися формулировками являются мультипликативная и аддитивная схемы. Мультипликативная обновляет приближение последовательно подобласть за подобластью, используя на шаге i свежие значения с уже обработанной подобласти $i - 1$. Аддитивная схема выполняет шаги на всех подобластях независимо, исходя из общего предыдущего приближения, а в зонах перекрытия итог усредняется [5].

Каждая из схем имеет собственные сильные стороны. Аддитивная очевидным образом ложится на параллельное исполнение, поскольку локальные задачи решаются независимо [6]. Мультипликативная требует меньше итераций для достижения той же точности, однако последовательный порядок обновления затрудняет её распараллеливание [7]. Какой из подходов

окажется выгоднее на конкретной задаче, заранее предсказать сложно: ответ зависит от размера сетки, числа подобластей, ширины зоны перекрытия и характеристик используемого оборудования.

Целью настоящей работы является реализация и сравнение мультипликативной и аддитивной схем метода Шварца применительно к стационарной задаче, описываемой уравнением энергии в одно- и двумерной постановках.

Для её достижения были поставлены следующие задачи.

1. Получить разностные соотношения для одномерной и двумерной задачи, обосновать выбор шаблона и порядка аппроксимации.
2. Разработать программный код решателя на языке MATLAB, разделив его на функции согласно подзадачам (построение матриц, шаг метода, головной цикл).
3. Подготовить интерактивные приложения для запуска экспериментов и визуализации результатов.
4. Исследовать зависимость качества сходимости от размера области пересечения, числа подобластей и распределения узлов сетки.
5. Сопоставить аддитивную и мультипликативную схемы по числу итераций и реальному времени работы.
6. Проверить идею ускорения за счёт замены прямого решения на подобласти итерационным методом Зейделя.

Литература по методам декомпозиции области достаточно обширна. Базовая теория и условия сходимости мультипликативной и аддитивной схем подробно изложены в монографии Тоселли и Видлунда [3], а также в работе Куартерони и Валли [5]. В обоих источниках обоснована роль перекрытия подобластей: чем больше радиус пересечения, тем быстрее сходимость, а конкретный вид оценки константы сходимости зависит от геометрии и порядка аппроксимации.

Вопросам параллельной организации вычислений посвящена работа Ортеги [7]. Особый интерес для практической части представляет обсуждение накладных расходов, возникающих при дроблении задачи на большое

число подобластей. Современная трактовка аддитивной схемы как предобуславливателя приведена у Кая и Саркиса [6]; авторы показывают, что ограничение зоны перекрытия позволяет сохранить качество сходимости при существенном снижении объёма пересылок между процессами.

Поведение алгоритмов при малой ширине области пересечения изучено Дрейя и Видлундом [8]. В частности, показано, что слишком узкое перекрытие приводит к зависимости числа итераций от шага сетки, что существенно ограничивает применимость метода.

Разностные схемы для эллиптических уравнений, использованные в работе для аппроксимации лапласиана, изложены в [2] и [9]. Приёмы работы с плохо обусловленными системами, оказавшиеся полезными при рассмотрении метода Зейделя как локального решателя на подобласти, описаны в [10].

1 Постановка задачи

1.1 Краевая задача и разностная аппроксимация

В качестве отправной точки выбрана краевая задача для стационарного одномерного уравнения с известной правой частью:

$$\frac{d^2 u}{dx^2} = f(x), \quad x \in (a, b), \quad u(a) = u_a, \quad u(b) = u_b. \quad (1)$$

Здесь $f(x)$ — заданная функция, u_a и u_b — значения искомой функции на концах отрезка. Постановка (1) соответствует одномерному варианту уравнения теплопроводности в стационарном случае при нормировке, переводящей коэффициент теплопроводности в единицу.

Введём на отрезке $[a, b]$ сетку $a = x_0 < x_1 < \dots < x_n = b$ с неравномерным шагом $h_i = x_i - x_{i-1}$. Вторую производную аппроксимируем симметричной разностью, обеспечивающей второй порядок точности на неравномерной сетке [2]:

$$\left. \frac{d^2 u}{dx^2} \right|_{x_i} \approx \frac{2}{h_i + h_{i+1}} \left(\frac{u_{i+1} - u_i}{h_{i+1}} - \frac{u_i - u_{i-1}}{h_i} \right). \quad (2)$$

В частном случае равномерного разбиения $h_i \equiv h$ выражение упрощается до известной формы $u_{xx} \approx (u_{i+1} - 2u_i + u_{i-1})/h^2$. Перенос соседних узлов

в правую часть даёт пару рекуррентных соотношений, удобных при последовательном обходе подобласти как слева направо, так и справа налево:

$$\begin{cases} u_{i+1} = 2u_i - u_{i-1} + h^2 f_i, \\ u_{i-1} = 2u_i - u_{i+1} + h^2 f_i. \end{cases} \quad (3)$$

В обозначениях величины

$$c_i^{(L)} = \frac{2}{h_i(h_i + h_{i+1})}, \quad c_i^{(R)} = \frac{2}{h_{i+1}(h_i + h_{i+1})}, \quad c_i^{(M)} = -\frac{2}{h_i h_{i+1}} \quad (4)$$

играют роль элементов левой (`glob_lower`), правой (`glob_upper`) и главной (`glob_main`) диагоналей. Подстановка (2) в (1) с переносом граничных значений в правую часть даёт линейную систему $A u_{\text{int}} = \tilde{f}$ относительно вектора внутренних значений $u_{\text{int}} = (u_1, \dots, u_{n-1})^\top$. Матрица A имеет трёхдиагональную структуру и в случае равномерной сетки — симметрична:

$$A = \begin{pmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \cdots & 0 \\ 0 & 1 & -2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -2 \end{pmatrix}, \quad \tilde{f} = h^2 \begin{pmatrix} f_1 - u_a/h^2 \\ f_2 \\ \vdots \\ f_{n-1} - u_b/h^2 \end{pmatrix}. \quad (5)$$

Прямое решение этой системы используется как эталонное численное приближение, с которым в дальнейшем сопоставляются итерационные результаты метода Шварца.

1.2 Двумерная постановка

В двумерном случае рассматривается уравнение Пуассона на единичном квадрате $\Omega = (0, 1)^2$:

$$\Delta u(x, y) = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y), \quad (x, y) \in \Omega, \quad (6)$$

с граничными условиями Дирихле на $\partial\Omega$. В качестве модельной правой части выбрана функция $f(x, y) = \sin(\pi x) \sin(\pi y)$, краевые условия однородные, за исключением правой границы, где $u(1, y) = \sin(\pi y)$. Аналитическое

решение в такой постановке выражается через гиперболический синус:

$$u(x, y) = \left(\frac{\sinh(\pi x)}{\sinh(\pi)} - \frac{1}{2\pi^2} \sin(\pi x) \right) \sin(\pi y). \quad (7)$$

Введём прямоугольную сетку $x_0 < \dots < x_n$, $y_0 < \dots < y_m$ с шагами h_x и h_y . Стандартный пятиточечный шаблон даёт следующую аппроксимацию:

$$h_y^2 u_{i+1,j} + h_y^2 u_{i-1,j} + h_x^2 u_{i,j+1} + h_x^2 u_{i,j-1} - 2(h_x^2 + h_y^2) u_{i,j} = h_x^2 h_y^2 f_{i,j}. \quad (8)$$

При равных шагах форма (8) сводится к привычной записи $u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}$. На сетках с существенно различающимися шагами по осям использование единого значения h приводит к значительной потере точности — что было подтверждено при прогонке предварительных тестов.

Разворачивая внутренние значения u_{ij} в один вектор по столбцам, получаем разреженную систему с пятидиагональной матрицей порядка $(n-1)(m-1)$. Граничные значения переносятся в правую часть.

1.3 Идея метода Шварца

Принцип метода [4] состоит в следующем. Расчётная область Ω покрывается семейством пересекающихся подобластей $\Omega_1, \dots, \Omega_p$, причём $\bigcup_i \Omega_i = \Omega$. На каждой подобласти решается локальная задача того же типа, что и исходная (1) или (6). Условия на участках $\partial\Omega_i \setminus \partial\Omega$, лежащих внутри Ω , снимаются с соседних подобластей, в которых решение к данному моменту уже получено или известно с предыдущей итерации. Процесс повторяется до тех пор, пока приближения на соседних подобластях не согласуются с заданной точностью.

Для одномерной постановки на отрезке $[a, b]$ типичный вариант декомпозиции на две подобласти изображён на рисунке 1. Граничные точки ξ_1 и ξ_2 задают ширину зоны перекрытия; число узлов сетки, попадающих в неё, обозначается далее r и называется радиусом пересечения. В двумерном случае соответствующая картина выглядит как разбиение квадрата Ω на вертикальные полосы Ω_1 и Ω_2 , общая часть которых — полоса узлов, лежащих между ξ_1 и ξ_2 (рисунок 2).

Различие между мультипликативной и аддитивной схемами проявляется в порядке обновления подобластей. В мультипликативном варианте

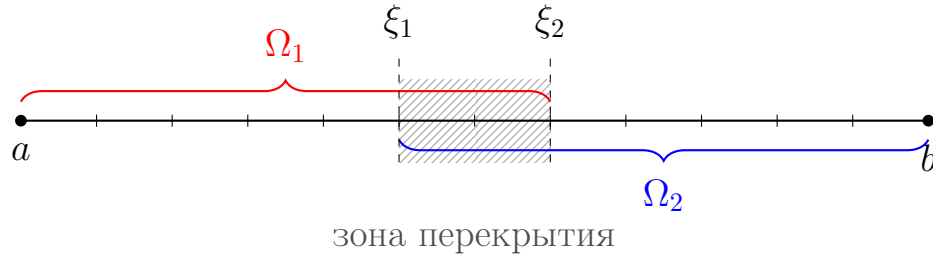


Рисунок 1 – Декомпозиция отрезка $[a, b]$ на две пересекающиеся подобласти Ω_1 и Ω_2 . Заштрихованная полоса от ξ_1 до ξ_2 соответствует общим для обеих подобластей узлам сетки

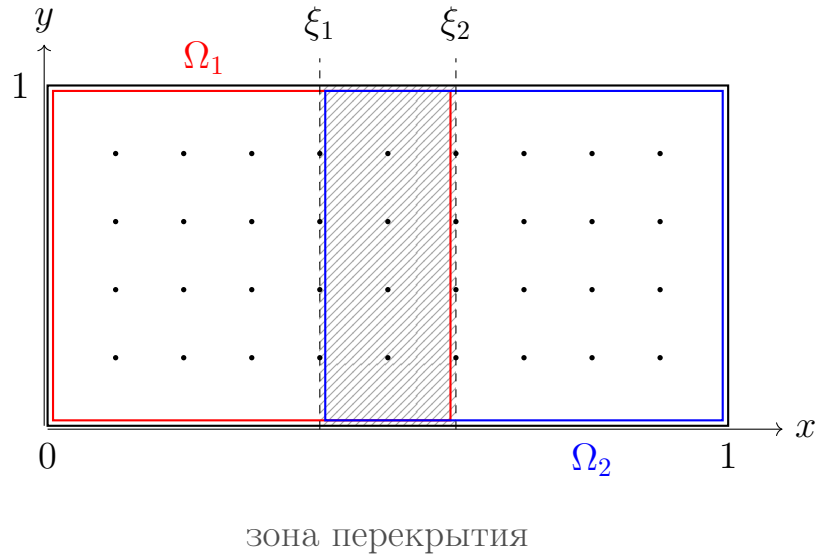


Рисунок 2 – Декомпозиция квадрата $\Omega = (0, 1)^2$ на две вертикальные подобласти Ω_1 и Ω_2 . Заштрихованная полоса соответствует узлам сетки, входящим в обе подобласти одновременно

обработка подобласти Ω_i использует уже пересчитанное на текущей итерации решение со всех Ω_j , $j < i$. Эта схема является непосредственным аналогом метода Гаусса—Зейделя для задачи на интерфейсе. В аддитивном варианте все локальные задачи на одной итерации решаются исходя из единого предыдущего приближения, после чего значения в перекрытых узлах усредняются по числу подобластей, которым этот узел принадлежит — по аналогии с методом Якоби [7].

1.4 Алгоритмы

Приведём оба алгоритма в виде, отражающем фактически реализованный код. Пусть подобласть Ω_i задаётся набором индексов внутренних узлов $I_i = \{s_i, s_i + 1, \dots, e_i\}$, где s_i и e_i — индексы первого и последнего

внутренних узлов подобласти в глобальной нумерации. Сосед слева имеет индекс $s_i - 1$ (узел с уже известным значением), сосед справа — индекс $e_i + 1$. Через A_i обозначим блок матрицы A , соответствующий индексам I_i ; через f_i — сужение правой части $\tilde{f}$ на эти индексы. Параметры $c_i^{(L)}, c_i^{(R)}$ из (4) задают вклады левой и правой границ соответственно.

В мультипликативной схеме обновление подобластей выполняется последовательно. Пусть $u^{(k)} \in \mathbb{R}^{n+1}$ — приближение к решению на k -й итерации, причём граничные значения $u_0^{(k)} = u_a$ и $u_n^{(k)} = u_b$ фиксированы. Тогда переход к следующему приближению описывается итерацией:

$$\begin{aligned} u^{(k+1)} &\leftarrow u^{(k)}; \\ \text{для } i = 1, 2, \dots, p : \\ f_i^* &= f_i; \quad f_i^*(s_i) \leftarrow c_{s_i}^{(L)} u_{s_i-1}^{(k+1)}; \quad f_i^*(e_i) \leftarrow c_{e_i}^{(R)} u_{e_i+1}^{(k+1)}; \\ u_{I_i}^{(k+1)} &\leftarrow A_i^{-1} f_i^*. \end{aligned} \tag{9}$$

Ключевая особенность — обращение к уже обновлённым значениям $u^{(k+1)}$ на левой и правой границах подобласти, что и обеспечивает порядок Гаусса—Зейделя для интерфейсных значений.

В аддитивной схеме все локальные задачи решаются исходя из единого предыдущего приближения, после чего значения в перекрытых узлах усредняются по числу подобластей, к которым относится данный узел:

$$\begin{aligned} S &\leftarrow 0, \quad W \leftarrow 0; \\ \text{(возможно параллельно) для } i = 1, 2, \dots, p : \\ f_i^* &= f_i; \quad f_i^*(s_i) \leftarrow c_{s_i}^{(L)} u_{s_i-1}^{(k)}; \quad f_i^*(e_i) \leftarrow c_{e_i}^{(R)} u_{e_i+1}^{(k)}; \\ \tilde{u}_i &\leftarrow A_i^{-1} f_i^*; \quad S(I_i) \leftarrow S(I_i) + \tilde{u}_i; \quad W(I_i) \leftarrow W(I_i) + 1; \\ \text{затем для } j \in \bigcup_i I_i : \quad u_j^{(k+1)} &= S_j / W_j. \end{aligned} \tag{10}$$

В неперекрытых узлах $W_j = 1$, и значение совпадает с локальным решением $\tilde{u}_i$. В перекрытых $W_j \geq 2$, и итоговое значение получается как среднее арифметическое решений на смежных подобластях.

Критерий остановки в обеих схемах основан на максимальном по мо-

дулю изменении приближения за итерацию по всем внутренним узлам:

$$\varepsilon^{(k)} = \max_{i=1,\dots,n-1} |u_i^{(k+1)} - u_i^{(k)}| < \varepsilon_{\text{tol}}. \quad (11)$$

Алгоритмы (9)–(10) непосредственно переносятся на двумерный случай: под-области соответствуют столбцам сеточной функции, локальная матрица A_i строится пятиточечным шаблоном для выбранного диапазона столбцов, а перенос значений на границах выполняется по столбцам $s_i - 1$ и $e_i + 1$. Реализации обеих схем приведены в листингах А.1 и Б.1.

1.5 Модельный пример

Для контроля корректности численного решения использовались две тестовые конфигурации. В одномерной задаче бралось $f(x) = \sin(x)$; этой правой части соответствует точное решение $u(x) = -\sin(x)$. Рассматривались отрезки $[0, \pi]$ и $[0, 2\pi]$. В первом случае краевые условия однородны, во втором — включают одно ненулевое промежуточное значение:

$$\left[\begin{array}{l} [0, 2\pi] : \\ [0, \pi] : \end{array} \right. \left\{ \begin{array}{l} u(0) = 0, \\ u(\pi) = 0, \\ u(2\pi) = 0, \\ u(0) = 0, \\ u(\frac{\pi}{2}) = 1, \\ u(\pi) = 0. \end{array} \right. \quad (12)$$

Второй вариант представляет дополнительный интерес: внутреннее значение решения отлично от нуля, что облегчает выявление ошибок, связанных с неточным переносом граничных условий между подобластями.

В двумерной задаче, как уже отмечалось ранее, использовалось уравнение Пуассона с правой частью $\sin(\pi x) \sin(\pi y)$ и неоднородным граничным условием $u(1, y) = \sin(\pi y)$, для которого аналитическое решение задано формулой (7).

2 Практическая часть

2.1 Программный комплекс

Расчёты выполнялись в среде MATLAB [11]. Исходный код проекта доступен в открытом репозитории [12]. Структура кода сознательно сделана модульной: каждая логически отдельная операция вынесена в самостоятельную функцию. Это упрощает отладку и поддержку, а также позволяет переиспользовать общие части. Например, функция построения разреженной двумерной матрицы по коэффициентам сетки (`build_2d_matrix.m`) применяется и при формировании эталонного решателя на всей области, и при формировании локальных матриц подобластей в методе Шварца.

Каталог проекта организован следующим образом:

- `solvers/` — функции построения матриц, шаги мультипликативного и аддитивного методов, прямые решатели для одно- и двумерной задачи;
- `scripts/` — головные циклы методов, объединяющие инициализацию, основной цикл итераций и сбор истории сходимости;
- `apps/` — два интерактивных приложения, построенных на `uifigure` и предназначенных для проведения экспериментов с интерактивной настройкой параметров;
- `utils/` — вспомогательные функции, среди которых `warped_linspace.m` для построения сетки с управляемой плотностью узлов.

Базовый цикл одномерного метода представлен в приложении Г (листинг Г.1). Указатель `method` ссылается либо на функцию `schwartz_mult_step`, либо на `schwartz_add_step`; переключение между ними определяется логическим параметром `is_mult`. Это позволяет запускать сравнение двух схем при полностью идентичных входных данных.

2.2 Одномерная задача

Эксперименты с одномерной задачей были начаты с верификации корректности. Сначала рассматривалась задача на $[0, \pi]$ с правой частью $\sin(x)$

и однородными краевыми условиями. Эталонное решение, полученное прямой прогонкой системы (5), сопоставлялось с приближением, выдаваемым методом Шварца после сходимости.

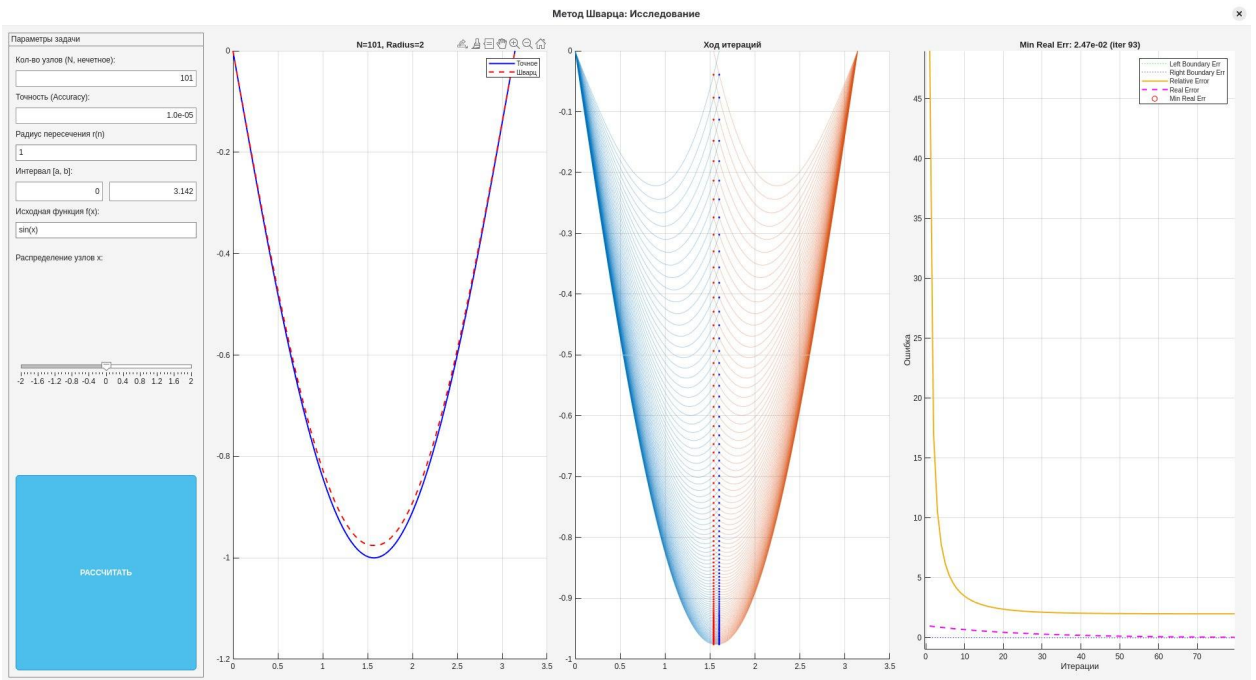


Рисунок 3 – Одномерное приложение: слева — сравнение точного решения и решения по Шварцу; в центре — история итераций по подобластям; справа — графики оценочной и фактической ошибок

Интерфейс приложения (рисунок 3) объединяет три типа графиков. На левой панели отображается совпадение точного и приближённого решений после сходимости. На центральной — ход итераций по каждой подобласти, причём цвет графика соответствует её номеру. На правой — сопоставление оценки ошибки по критерию (11) с фактической, посчитанной относительно эталона. Расхождение между ними позволяет судить о надёжности оценочного критерия остановки.

Первый результат соответствует ожиданиям: мультипликативная схема сходится быстрее аддитивной на той же сетке и при одинаковых параметрах. Это согласуется с известным результатом из [3]: аддитивный шаг интерпретируется как метод Якоби для значений на интерфейсе между подобластями, а мультипликативный — как Зейделя для той же системы. Отдельный экспериментальный вариант аддитивной схемы с параллельным запуском локальных задач средствами `parfor` оформлен в файле `schwartz_add_step_cont.m`, однако в одномерной задаче он не даёт выигрыша:

накладные расходы на создание параллельных задач превышают выгоду от одновременного исполнения. В основном расчёте 1D используется последовательный обход подобластей.

2.3 Размер области пересечения

Один из управляющих параметров метода — радиус пересечения подобластей r , выраженный в числе узлов сетки. Первоначальное предположение состояло в том, что разумно брать r небольшим, экономя на работе с дублирующими узлами. Эксперимент показал, что это представление ошибочно.

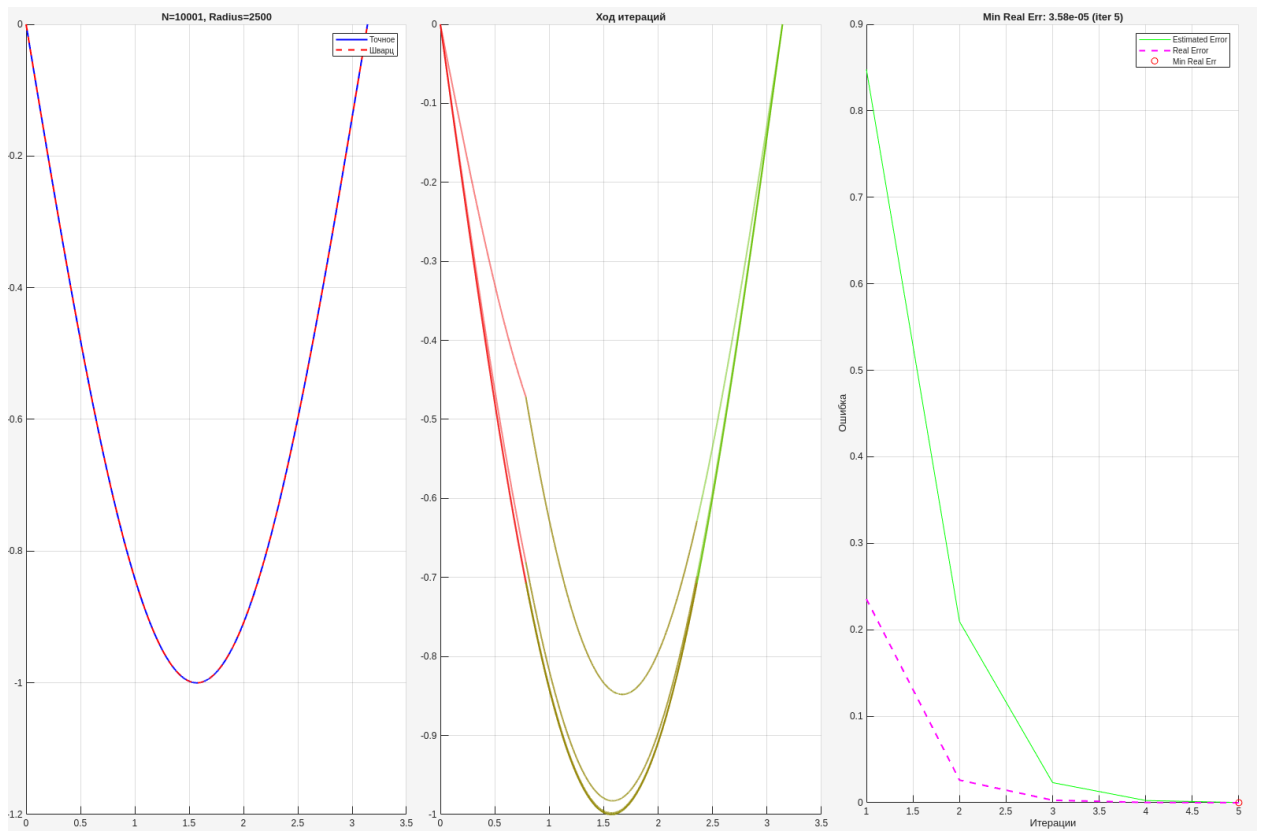


Рисунок 4 – Случай $N = 10001$, радиус пересечения $r = n/4$. Сходимость достигается за 5 итераций

При выборе r равным примерно четверти числа узлов сетки ($r \approx n/4$) число итераций до сходимости перестаёт зависеть от n и в большинстве проведённых тестов оказывается на уровне 4–5 (рисунок 4). Дальнейшее увеличение r снижает это значение ещё сильнее: максимальное ускорение, зафиксированное в экспериментах, достигается при $r \approx 3n/4$, когда метод сходится за две итерации. Однако такое перекрытие близко к полному

дублированию данных, и выгоды по сравнению с прямым решением задачи на всей области не наблюдается.

Уменьшение r даёт качественно противоположную картину. При $r = 1$ или $r = 2$ число итераций растёт с увеличением n , что согласуется с теоретической оценкой константы сходимости для метода Шварца с малым перекрытием [8].

2.4 Распределение узлов сетки

Если сетка равномерная и общее число узлов невелико, попытка увеличить r наталкивается на ограничение: подобласти становятся практически одинаковыми и смысл декомпозиции теряется. Альтернативный путь — сгустить узлы у концов отрезка, разредив центральную часть. Тогда фиксированному значению r соответствует более широкая зона перекрытия, при том же общем числе неизвестных. Реализация представлена в файле `warped_linspace.m` (листинг В.1):

$$\tilde{x}(t) = a + (b - a) \cdot \frac{\operatorname{sgn}(t) |t|^{e^k} + 1}{2}, \quad t \in [-1, 1]. \quad (13)$$

Параметр k управляет распределением: при $k = 0$ получается равномерная сетка ($p = e^0 = 1$), при $k > 0$ узлы смещаются к концам, а центральная часть разреживается. Чем глубже провал плотности в середине, тем больший относительный размер по числу узлов имеет зона перекрытия.

Эффект оказался выраженным. На сетке со сгущением узлов у концов отрезка при том же $r = 2$ метод сходится за 9 итераций (рисунок 5) против более сотни на равномерной сетке. По существу получен косвенный путь расширить эффективное перекрытие, не увеличивая радиус r в явном виде: достаточно сделать сами узлы в зоне перекрытия более разрежёнными относительно концов.

2.5 Количество подобластей

Поведение метода при изменении числа подобластей p оказалось менее очевидным, чем ожидалось. В проведённых экспериментах оптимум по совокупному времени работы достигается при $p = 2$, а дальнейшее наращивание до $p = 4$ или $p = 5$ даёт заметное замедление. Причины такого поведения

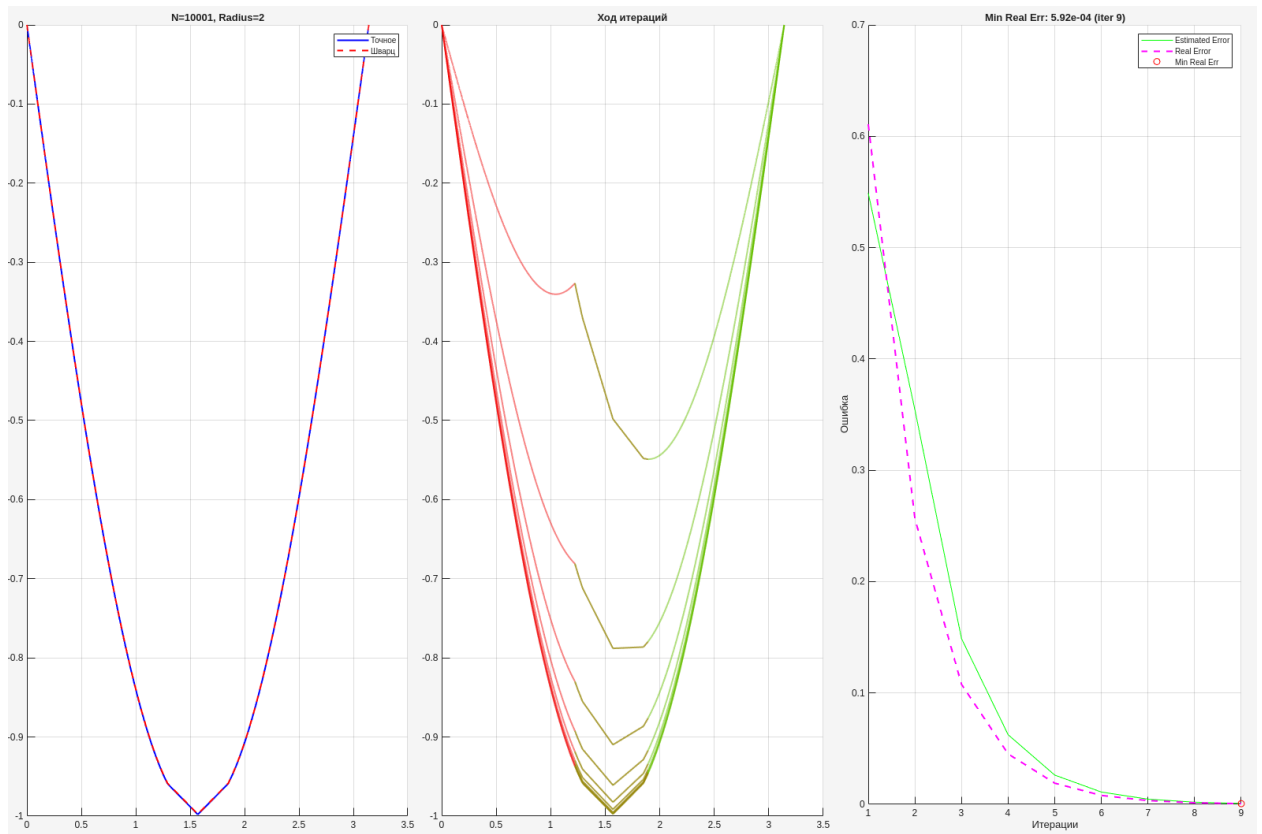


Рисунок 5 – Случай $N = 10001$, $r = 2$ при сгущении сетки к концам отрезка. Сходимость достигается за 9 итераций вместо нескольких сотен на равномерной сетке

следующие:

- с ростом p возрастает число итераций до сходимости, поскольку на одной итерации информация проходит меньшее расстояние по сетке;
- уменьшение размера локальных задач не компенсирует увеличение их числа — особенно с учётом фиксированных накладных расходов на формирование локальных правых частей и распределение данных в `parfor`;
- для качественного выигрыша от параллельной аддитивной схемы требуется размер локальной задачи, превышающий некоторый порог; ниже этого порога последовательный код эффективнее.

На рисунке 6 приведён ход итераций при $p = 5$: неустойчивость в зоне перекрытий выражена сильнее, и количество итераций до сходимости заметно превосходит наблюдаемое при $p = 2$.

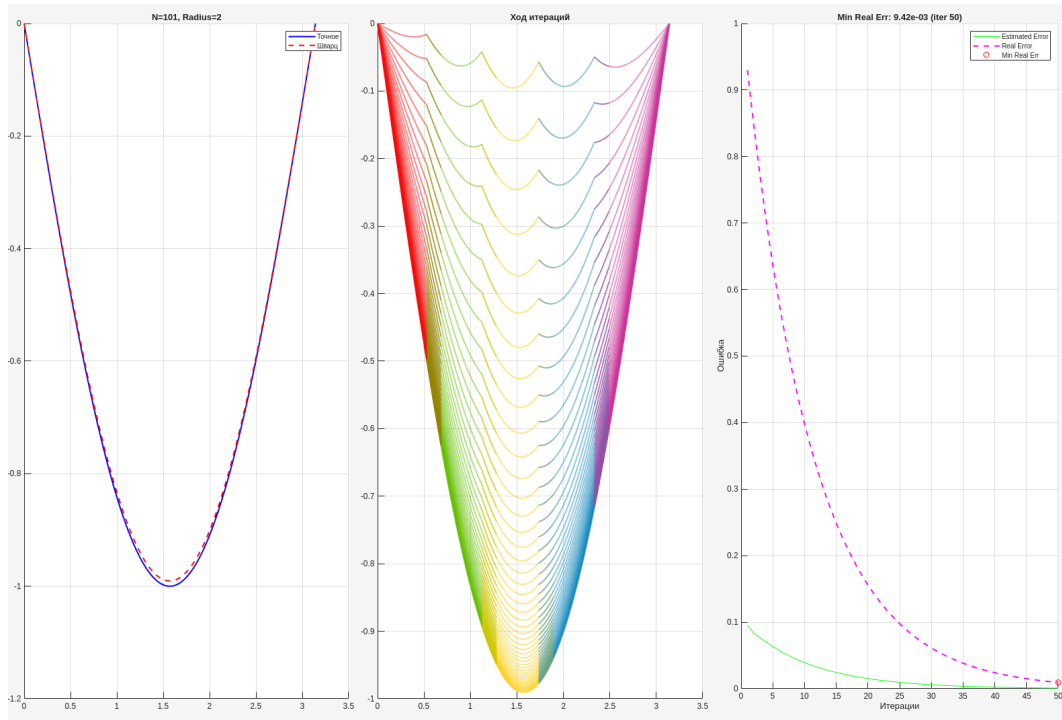


Рисунок 6 – Разбиение на 5 подобластей. Итераций до сходимости требуется существенно больше, чем при двух подобластях; выигрыша по реальному времени работы не наблюдается

2.6 Метод Зейделя как локальный решатель

Поскольку основная вычислительная нагрузка на одной итерации приходится на решение трёхдиагональных систем на подобластях, представляет интерес замена прямого решателя итерационным с ограничением числа шагов. В работе реализован метод Зейделя — функция `solvers/seidel.m` с параметрами допуска `tol` и предельного числа внутренних итераций `max_iter`.

Скрипт `seidel_schwartz_1d_symmetrical.m` использует метод Зейделя для локального решения на каждой подобласти. Была выдвинута гипотеза о том, что на каждом внешнем шаге метода Шварца достаточно получать грубое приближение, а его уточнение происходит за счёт самой итерации декомпозиции.

На практике обнаружился эффект, противоречащий исходной гипотезе. После нескольких внешних итераций метод выходит на медленный режим, а при слишком слабом задании внутренней точности `tol` в зоне перекрытий наблюдается немонотонная сходимость с резкими отклонениями приближения (рисунок 7). Выигрыш на отдельных запусках есть, но

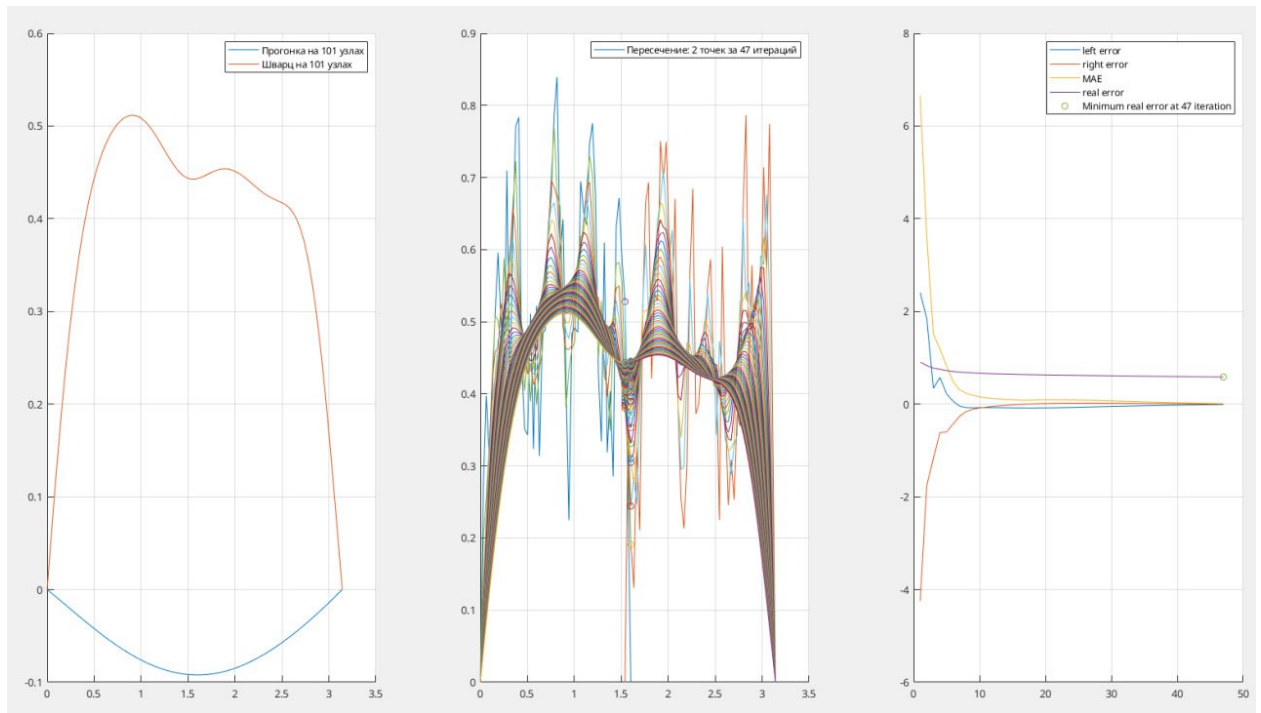


Рисунок 7 – Применение метода Зейделя в качестве локального решателя. При недостаточной точности внутренних итераций пропадает монотонность сходимости, и метод может расходиться

он неустойчив. Согласованное управление двумя точностями — внешней и внутренней — оказалось практически неудобным, и в общем виде идея себя не оправдала.

2.7 Двумерная задача

Перенос алгоритма в двумерный случай выполнен по той же схеме: каждая подобласть представляет собой вертикальную полосу в плоскости (x, y) , а на интерфейсе переносятся столбцы граничных значений. Локальная матрица строится функцией `build_2d_matrix.m` в виде разреженной пятидиагональной матрицы.

В качестве модельного примера использовалось уравнение Пуассона (6) с правой частью $\sin(\pi x) \sin(\pi y)$ и неоднородным правым граничным условием $u(1, y) = \sin(\pi y)$, для которого аналитическое решение записывается формулой (7). Поскольку точное решение известно, оно и использовалось как основной эталон при оценке погрешности. Дополнительно для контроля чисто разностной составляющей ошибки применялся полный решатель `solve2d.m`, который не использует декомпозицию и собирает разреженную систему сразу для всей области.

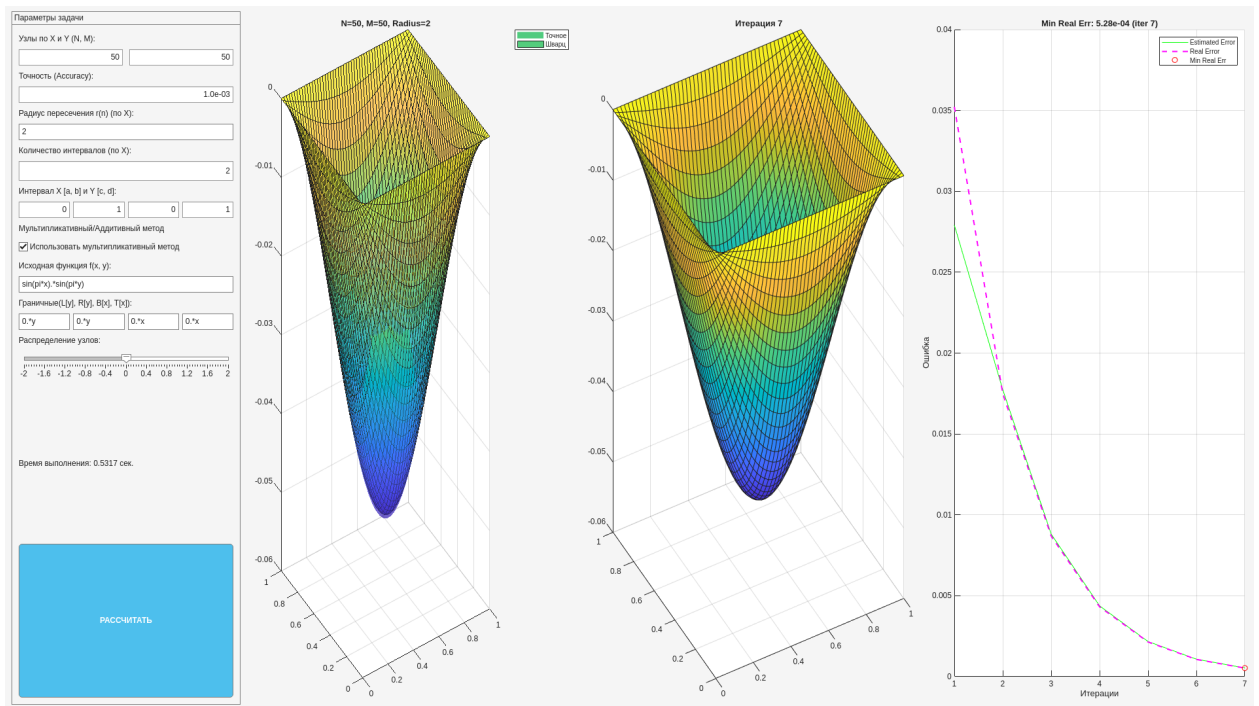


Рисунок 8 – Двумерное приложение. Слева — панель параметров, в центре — сравнение точного и приближённого решений в виде поверхностей, далее — ход итераций и графики ошибки

Качественные выводы из одномерной задачи переносятся и сюда: оптимум по числу подобластей по-прежнему достигается при $p = 2$, а зависимость поведения метода от радиуса перекрытия имеет тот же характер. Существенное различие появляется при сравнении схем по реальному времени работы. На сетках до примерно 300×300 узлов мультипликативная схема сохраняет преимущество: быстрая сходимость перекрывает накладные расходы. При большем размере локальные задачи становятся достаточно крупными, чтобы накладные расходы на `parfor` перестали играть решающую роль; в этом режиме аддитивный вариант начинает превосходить мультипликативный по времени работы.

2.8 Сводка результатов

Если объединить наблюдения в общий список, картина получается следующей.

- Оптимальное число подобластей по времени работы — $p = 2$ для обеих размерностей задачи.
- Оптимальное перекрытие по числу итераций — $r \approx n/4$, при этом

число итераций перестаёт зависеть от размера сетки.

- Сгущение сетки у концов отрезка даёт ускорение в десятки раз при том же физическом размере перекрытия.
- Мультипликативная схема устойчиво выигрывает в одномерной задаче и в двумерной задаче на сетках умеренного размера.
- Аддитивная схема превосходит мультипликативную начиная с сеток приблизительно 300×300 узлов в двумерной задаче за счёт параллельного запуска локальных решений.
- Использование метода Зейделя как локального решателя сокращает время одной итерации, но в общем случае подрывает устойчивость внешней сходимости.

ЗАКЛЮЧЕНИЕ

В рамках выполненной работы реализованы и сопоставлены мультипликативная и аддитивная схемы метода Шварца применительно к стационарной краевой задаче в одно- и двумерной постановке. Полученные экспериментальные результаты согласуются с известными теоретическими оценками из [3, 5] и одновременно дают вполне конкретные практические рекомендации: дробление задачи более чем на две подобласти оказывается невыгодным, оптимальное перекрытие соответствует приблизительно $n/4$ узлам, а сгущение узлов у концов отрезка позволяет ускорить сходимость без увеличения общего числа неизвестных. Аддитивная схема становится конкурентной по реальному времени лишь при достаточно больших размерах локальных задач: на сетках небольшого размера её выигрыш поглощается накладными расходами на параллельный запуск.

Отдельно рассмотрена возможность использования метода Зейделя в качестве локального решателя на подобластях. В общем виде подход оказался неустойчивым: ослабление внутренней точности приводит к потере монотонности внешней сходимости, метод выходит на плато либо начинает расходиться. Возвращение к этой идее имеет смысл при использовании более устойчивых локальных схем, например многосеточных, либо при адаптивном управлении внутренней точностью.

В качестве направлений продолжения работы можно отметить декомпозицию по оси большей протяжённости или одновременно по двум осям, использование условий третьего рода на интерфейсе подобластей с целью повышения робастности, а также адаптацию метода к нестационарному уравнению теплопроводности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Тихонов, А. Н. Уравнения математической физики / А. Н. Тихонов, А. А. Самарский. — 6-е изд. — Москва : Изд-во МГУ, 1999. — 799 с.
2. Самарский, А. А. Теория разностных схем / А. А. Самарский. — Москва : Наука, 1989. — 616 с.
3. Toselli, A. Domain Decomposition Methods: Algorithms and Theory / A. Toselli, O. Widlund. — Berlin : Springer, 2005. — 450 с.
4. Schwarz, H. A. Über einen Grenzübergang durch alternierendes Verfahren / H. A. Schwarz // Vierteljahrsschrift der Naturforschenden Gesellschaft in Zürich. — 1870. — Т. 15. — С. 272—286.
5. Quarteroni, A. Domain Decomposition Methods for Partial Differential Equations / A. Quarteroni, A. Valli. — Oxford : Oxford University Press, 1999. — 376 с.
6. Cai, X.-C. A Restricted Additive Schwarz Preconditioner for General Sparse Linear Systems / X.-C. Cai, M. Sarkis // SIAM Journal on Scientific Computing. — 1999. — Т. 21, № 2. — С. 792—797.
7. Ортега, Д. Введение в параллельные и векторные методы решения линейных систем / Д. Ортега. — Москва : Мир, 1991. — 367 с.
8. Dryja, M. Domain Decomposition Algorithms with Small Overlap / M. Dryja, O. B. Widlund // SIAM Journal on Scientific Computing. — 1994. — Т. 15, № 3. — С. 604—620.
9. Бахвалов, Н. С. Численные методы / Н. С. Бахвалов, Н. П. Жидков, Г. М. Кобельков. — 3-е изд. — Москва : БИНОМ. Лаборатория знаний, 2003. — 632 с.
10. Голуб, Д. Матричные вычисления / Д. Голуб, Ч. Ван Лоун. — Москва : Мир, 1999. — 548 с.
11. The MathWorks, Inc. MATLAB Documentation / The MathWorks, Inc. — 2023. — URL: <https://www.mathworks.com/help/matlab/> (дата обр. 21.05.2026).

12. Лещенко, И. Ф. Schwartz: реализация аддитивного и мультипликативного методов Шварца на MATLAB / И. Ф. Лещенко. — 2025. — URL: <https://github.com/rokokol/schwartz> (дата обр. 22.05.2026).

ПРИЛОЖЕНИЕ А (обязательное)

Мультипликативный шаг метода Шварца

Листинг А.1 – Мультипликативный шаг (solvers/schwartz_mult_step.m)

```
1 function [u_new, max_diff] = schwartz_mult_step(u_old, A_cells,  
2     f_int, starts, ends_arr, glob_lower, glob_upper, bpoints)  
3     u_new = u_old;  
4  
5     for i = 1:bpoints  
6         idx_int = starts(i) : ends_arr(i);  
7         f_local = f_int(idx_int);  
8  
9         left_u_val = u_new(starts(i));  
10        f_local(1) = f_local(1) - glob_lower(idx_int(1)) *  
11            left_u_val;  
12  
13        right_u_val = u_new(ends_arr(i) + 2);  
14        f_local(end) = f_local(end) - glob_upper(idx_int(end))  
15            * right_u_val;  
16  
17        u_local = A_cells{i} \ f_local;  
18  
19        u_new(idx_int + 1) = u_local;  
20    end  
  
21    max_diff = max(abs(u_new(2:end-1) - u_old(2:end-1)));  
22 end
```

ПРИЛОЖЕНИЕ Б (обязательное)

Аддитивный шаг метода Шварца

Листинг Б.1 – Аддитивный шаг (solvers/schwartz_add_step.m)

```
1 function [u_new, max_diff] = schwartz_add_step_cont(u_old, A_cells, f_int,
2 starts, ends_arr, glob_lower, glob_upper, bpoints)
3
4 parfor i = 1:bpoints
5     idx_int = starts(i) : ends_arr(i);
6     f_local = f_int(idx_int);
7
8     left_u_val = u_old(starts(i));
9     f_local(1) = f_local(1) - glob_lower(idx_int(1)) * left_u_val;
10
11     right_u_val = u_old(ends_arr(i) + 2);
12     f_local(end) = f_local(end) - glob_upper(idx_int(end)) *
        right_u_val;
13
14     u_locals{i} = A_cells{i} \ f_local;
15 end
16
17 u_new_sum = zeros(size(u_old));
18 weights = zeros(size(u_old));
19
20 for i = 1:bpoints
21     idx_int = starts(i) : ends_arr(i);
22     u_new_sum(idx_int + 1) = u_new_sum(idx_int + 1) + u_locals{i};
23     weights(idx_int + 1) = weights(idx_int + 1) + 1;
24 end
25
26 u_new = u_old;
27 mask = weights > 0;
28 u_new(mask) = u_new_sum(mask) ./ weights(mask);
29
30 max_diff = max(abs(u_new(2:end-1) - u_old(2:end-1)));
31 end
```


ПРИЛОЖЕНИЕ В (обязательное)

Нелинейное распределение узлов сетки

Листинг В.1 – Нелинейная сетка (utils/warped_linspace.m)

```
1 function res = warped_linspace(a, b, n, k)
2     x = linspace(-1, 1, n);
3     p = exp(k);
4     warped_x = sign(x) .* (abs(x) .^ p);
5
6     res = a + (b - a) * (warped_x + 1) / 2;
7 end
```

ПРИЛОЖЕНИЕ Г (обязательное)

Основной итерационный цикл одномерного метода

Листинг Г.1 – Основной цикл (фрагмент scripts/schwartz_1d_symmetrical.m)

```
1 flag = true;
2 while flag
3     iter = iter + 1;
4
5     [u_next, max_diff] = method(u_curr, A, f_int, ...
6                               starts, ends_arr, ...
7                               glob_lower, glob_upper, bpoints
8                               );
9
10    e      = [e;      max_diff];
11    U      = [U;      u_next' ];
12    u_curr = u_next;
13
14    flag = max_diff > accuracy && iter < 100;
15 end
```